QA Copilot User Testing Documentation

Purpose

This document explains how to verify that QA Copilot is working correctly from a user and admin perspective. It is intended for demos, handoff, internal QA, and client walkthroughs.

Application Overview

QA Copilot is a full-stack QA assistance platform with:

- Next.js frontend
- Express backend API
- PostgreSQL database through Prisma
- JWT-based authentication
- Admin approval workflow
- AI-assisted QA utilities such as test case generation, bug analysis, release risk analysis, API testing, content match, and design match

Environments and URLs

Default local URLs:

- Frontend: ‘http://localhost:3000‘
- API: ‘http://localhost:4000‘
- API health check: ‘http://localhost:4000/api/health‘

Required Prerequisites

Before testing, confirm the following are available:

- Node.js and npm installed
- PostgreSQL database available
- ‘.env‘ configured correctly
- Dependencies installed

Minimum environment values:

‘‘‘env
DATABASE_URL=postgresql://...
JWT_SECRET=your-secret
OPENAI_API_KEY=your-openai-key
OPENAI_MODEL=gpt-4.1-mini
NEXT_PUBLIC_APP_URL=http://localhost:3000

NEXT_PUBLIC_API_URL=http://localhost:4000/api
API_HOST=0.0.0.0
API_PORT=4000
CORS_ORIGIN=http://localhost:3000
‘‘‘

Notes:

- AI features require a valid ‘OPENAI_API_KEY‘.
- Stripe values are optional for local testing.
- If Stripe is not configured, subscription updates fall back to a local update flow.

Initial Setup

Run the following commands from the project root:

‘‘‘powershell
npm install
npm run db:push
npm run db:seed
npm run dev
‘‘‘

Seeded Admin Account

The seed script creates a default admin account:

- Email: ‘admin@qacopilot.ai‘
- Password: ‘Admin@123‘

This account is used to approve newly registered users and validate the admin console.

Test Scope

The main test scope includes:

- Landing page availability
- User registration
- Pending approval handling
- Admin approval flow
- Approved user login
- Dashboard loading
- Project creation
- AI workflow execution

- Export functionality
- Billing fallback behavior

Smoke Test Checklist

Use this short checklist to quickly confirm the application is alive.

1. Open ‘http://localhost:3000‘
2. Confirm the landing page loads correctly
3. Open ‘http://localhost:4000/api/health‘
4. Confirm the API response is ‘{"status":"ok"}‘
5. Login with the admin account
6. Confirm the admin console opens successfully

If all items pass, the application is running at a basic operational level.

End-to-End User Test Cases

Test Case 1: Landing Page Availability

Steps:

1. Open ‘http://localhost:3000‘
2. Verify the hero section and navigation buttons are visible
3. Click ‘Start free trial‘
4. Click ‘Login‘

Expected Result:

- Landing page loads without errors
- ‘Start free trial‘ opens ‘/register‘
- ‘Login‘ opens ‘/login‘

Test Case 2: Register a New User

Sample data:

- Full name: ‘Test User‘
- Email: any new unused email address
- Password: ‘TestUser@123‘

Steps:

1. Open ‘/register‘

2. Fill the registration form
3. Submit the form

Expected Result:

- Registration completes successfully
- User is redirected to ‘/login?registered=1‘
- The screen shows a message that admin approval is required

Test Case 3: Validate Pending Approval Restriction

Steps:

1. Attempt to log in with the newly created user before approval

Expected Result:

- Login is blocked
- User sees a pending approval message

Test Case 4: Admin Approval Workflow

Steps:

1. Login using the admin account
2. Open ‘/admin‘
3. Locate the newly registered user
4. Approve the user

Expected Result:

- Admin dashboard loads successfully
- New user is visible in the user operations area
- Approval action completes successfully
- User status changes from pending to approved

Optional validation:

- Assign additional credits
- Confirm updated credit balance is reflected

Test Case 5: Approved User Login

Steps:

1. Logout from the admin account
2. Login with the approved user account

Expected Result:

- User is redirected to ‘/dashboard‘
- Dashboard loads successfully
- Credits, actions, and recent activity are visible

Test Case 6: Project Creation

Sample data:

- Project name: ‘Sample QA Project‘
- Description: ‘Testing application flow‘

Steps:

1. Open the dashboard
2. Create a new project

Expected Result:

- Project is created successfully
- Project appears in the project workspace
- Project can be selected for AI actions

Test Case 7: AI Workflow Validation

Recommended first workflow: ‘Bug Analysis‘

Sample input:

‘‘‘text
Login fails for some users after password reset.
Observed behavior: API returns 401 after reset.
Possible clues: token mismatch, stale session, or password hash issue.
‘‘‘

Steps:

1. Open the ‘Bug Analysis‘ section
2. Paste the sample input

3. Run the action

Expected Result:

- Request completes successfully
- Output is shown in the result panel
- Recent activity updates
- Used credits increase

Important:

- If the OpenAI key is missing or invalid, this test will fail even if the app itself is otherwise running correctly.

Test Case 8: File Upload Workflow

Test one of the following:

- Generate Test Cases
- Generate Test Report
- Generate API Tests
- Content Match
- Design Match

Steps:

1. Open one workflow
2. Upload a supported file
3. Run the action

Expected Result:

- File upload is accepted
- Processing completes without server error
- Structured output is shown
- Export actions are available when applicable

Test Case 9: Export Validation

After generating a result, test available exports such as:

- JSON
- CSV
- Excel

- PDF

Expected Result:

- File downloads successfully
- Downloaded file opens successfully
- Exported content matches the generated result

Test Case 10: Billing Fallback Validation

Condition:

- Stripe keys are left empty

Steps:

1. Trigger a plan change or billing flow if exposed in the UI

Expected Result:

- The application does not crash
- Subscription updates locally without redirecting to Stripe

API Validation Points

Useful endpoints for direct checks:

- ‘GET /api/health‘
- ‘POST /api/auth/register‘
- ‘POST /api/auth/login‘
- ‘GET /api/auth/me‘
- ‘GET /api/dashboard/overview‘
- ‘GET /api/projects‘
- ‘POST /api/projects‘
- ‘GET /api/admin/overview‘

Access rules:

- Most routes require authentication
- Dashboard, projects, billing, and AI routes require an approved user
- Admin routes require an approved admin

Pass Criteria

The application can be considered working when all of the following pass:

- Landing page loads
- API health endpoint responds successfully
- User registration works
- Pending users cannot access the dashboard
- Admin approval works
- Approved users can log in
- Dashboard loads correctly
- Project creation works
- At least one AI workflow succeeds
- At least one export succeeds

Common Failure Reasons

AI workflows fail

Possible causes:

- Missing ‘OPENAI_API_KEY‘
- Invalid OpenAI key
- Model access issue

Frontend loads but data does not

Possible causes:

- Incorrect ‘NEXT_PUBLIC_API_URL‘
- Backend not running
- CORS origin mismatch

Admin login fails

Possible causes:

- Seed script did not run
- Seed script ran against a different database

Registration works but dashboard or approval fails

Possible causes:

- Prisma schema not pushed
- Wrong ‘DATABASE_URL‘

- Backend cannot access PostgreSQL

Recommended Demo Flow

For live demonstrations, use this order:

1. API health check
2. Landing page
3. User registration
4. Admin login
5. User approval
6. Approved user login
7. Project creation
8. AI workflow execution
9. Export download

Document References

- [README.md](C:\Users\shaya\OneDrive\Desktop\Test Engine\README.md)
- [TESTING_GUIDE.md](C:\Users\shaya\OneDrive\Desktop\Test Engine\TESTING_GUIDE.md)
- [.env.example](C:\Users\shaya\OneDrive\Desktop\Test Engine\.env.example)
- [prisma/seed.js](C:\Users\shaya\OneDrive\Desktop\Test Engine\prisma\seed.js)